Synchronization & deadlock

examples

- Reader-writer problem
- Review of producer-consumer problem
- Ring buffer (queue) implementation with condition variables & semaphores
- Deadlock, livelock
- Dining philosopher problem & different solutions

Reader-writer problem

Readers: only reads the data and does not update/change the data

Writers: both reads and writes the data

- allow multiple readers to access the data at the same time when there is no writing.
- To avoid data corruption, only one thread at a time may modify (write) the data structure and no readers may be reading at that time.

Problem: how can we efficiently synchronize multiple readers and writers such that multiple readers can read together, but a writer gets exclusive access?

First attempt with mutex

```
void read() {
   lock(&m)
   // do read stuff
   unlock(&m)
}

void write() {
   lock(&m)
   // do write stuff
   unlock(&m)
}
```

- No data corruption!
- But, only one reader can access the data. Readers wait for other readers

Second attempt

```
void read() {
   while(writing) {/*spin*/}

   reading = 1
   // do read stuff
   reading = 0
}
void write() {
   while(reading || writing) {/*spin*/}

   writing = 1
   // do write stuff
   writing = 0
}
```

- Race condition: multiple readers and the writers at the same time

With condition variables

Condition ok to read

Readers can access/read when no writers

Condition ok to write

Writers can access/update/write when no readers or writers

- ★ Only one thread should manipulate state variables (reading, writing) at a time.
  - Critical section

Reader

```
read() {
    lock(&m)
    while (writing)
        cond_wait(&cv, &m)
    reading++;
    /* Read here! */
    reading--
    cond_signal(&cv)
    unlock(&m)
}
```

pthread_cond_wait performs 3 actions.

- it atomically unlocks the mutex
- and then sleeps
  - (until it is woken by pthread_cond_signal or pthread_cond_broadcast).
- **the awoken thread must re-acquire the mutex lock before returning.**

➔ Thus only one thread can actually be running inside the critical section.

Solution to Reader-Writer problem?

➔ if any writers are writing, a reader will enter the cond_wait.
- OK!

➔ Only one reader can read!
- because readers did not unlock the mutex.

A better version for multiple readers

```
read() {
    lock(&m)
    while (writing)
        cond_wait(&cv, &m)
    reading++;
    unlock(&m)

    /* Read here! */

    lock(&m)
    reading--
    cond_signal(&cv)
    unlock(&m)
}
```

Solution to Reader-Writer problem?

- ➔ if any writers are writing, a reader will enter the cond_wait.
  - ◆ OK!
- ➔ only one thread can be executing code inside the critical section at any time!
- ➔ Readers can read at the same time

Does this mean that a writer and reader could read and write at the same time?

- ◆ No!
  - **cond_wait** requires the thread re-acquire the mutex lock before returning.

Writer

```
write() {
    lock(&m);
    while (reading || writing)
        cond_wait(&cv, &m);
    writing++;

    /* Write here! */

    writing--;
    cond_signal(&cv);//or cond_broadcast
    unlock(&m);
}
```

Explanation

- This uses pthread_cond_signal. This will only wake up one thread.
    - If many readers are waiting for the writer to complete, only one sleeping reader will be awoken from their slumber.
- The reader and writer should use cond_broadcast so that all threads should wake up and check their while-loop condition.

Problem with this solution

- Starvation: If readers are constantly arriving then a writer will never be able to proceed **(the 'reading' count never reduces to zero)**.

Last attempt

implement a bounded-wait for the writer.

- If a writer arrives they will still need to wait for existing readers
- however future readers must be placed in a “holding pen” and wait for the writer to finish.

The plan is when a writer arrives, and before waiting for current readers to finish, register our intent to write by incrementing a counter ‘writer’

Final writer solution

```
writer() {
    lock(&m)
    writers++
    while (reading || writing)
        cond_wait(&cv, &m)
    writing++
    unlock(&m)
    // perform writing here
    lock(&m)
    writing--
    writers--
    cond_broadcast(&cv)
    unlock(&m)
}
```

- Readers wait for the writers

final reader solution

```
read() {
   lock(&m)
   /*check if any writers waiting*/
   while(writers)
       cond_wait(&cv,&m)
   /*  No need to wait while(writing here) because it only exits the above loop when writing is zero*/
   while(writing)
       cond_wait(&cv,&m)
   reading++
   unlock(&m)
      /* Read here! */
   lock(&m)
   reading--
   cond_signal(&cv)
   unlock(&m)
}
```

- Readers wait for the writers
  - Writers have higher priority

Review: consumer-producer problem

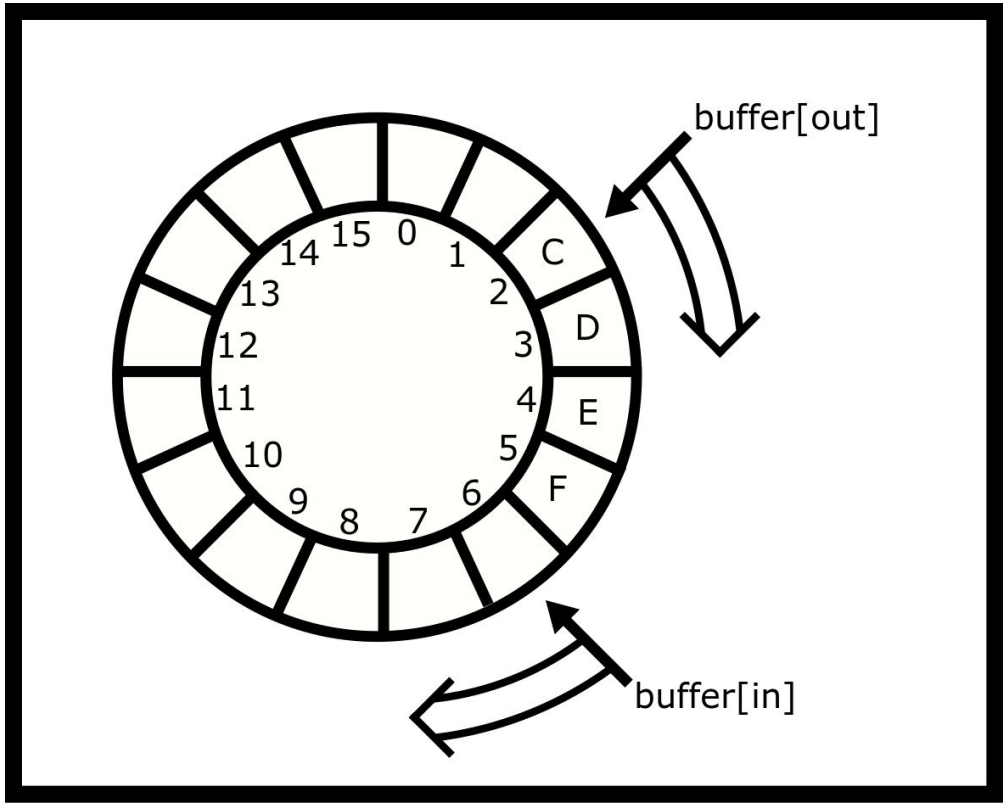
- a finite-size buffer
- and two classes of threads, producers and consumers,
  - producers: put items into the buffer
  - consumers: take items out of the buffer
- A producer must wait until the buffer has space before it can put something in,
- and a consumer must wait until something is in the buffer before it can take something out.

A **condition variable** represents a queue of threads waiting for some condition to be signaled.

Ring buffer

- A ring buffer is a simple, usually fixed-sized, storage mechanism
  - where contiguous memory is treated as if it is circular,
  - and two index counters keep track of the current beginning and end of the queue.
- As an array

```
void *buffer[16];
unsigned int in = 0, out = 0;
```



Ring buffer queue operations

```
void *buffer[16];
unsigned int in = 0, out = 0;

void enqueue(void *value) {
   buffer[in] = value;
   in++; /* Advance the index for next time */
   if (in == 16) in = 0; /* Wrap around! */
}
```

```
void *dequeue() {
   void *result = buffer[out];
   out++;
   if (out == 16) out = 0;
   return result;
}
```

- No control for buffer overflow/underflow!
- `if (in == 16) in = 0;`
  - `Using (in % 16) may cause problems if in is too big`
  - `b[ (in++) & (N-1) ] = value;`

Multithread solution: incorrect attempt

```
#define N 16
void *b[N]
int in = 0, out = 0;
p_m_t lock; sem_t s1,s2;
void init() {
   p_m_init(&lock, NULL);
   sem_init(&s1, 0, 16);
   sem_init(&s2, 0, 0);
}
enqueue(void *value) {
   p_m_lock(&lock)
   sem_wait( &s1 )
   b[ (in++) & (N-1) ] = value
   sem_post(&s1)
   p_m_unlock(&lock)
}
```

- The enqueue method waits and posts on the same semaphore (s1) and similarly with enqueue and (s2) i.e. we decrement the value and then immediately increment the value, so by the end of the function the **semaphore value is unchanged!**

- The initial value of s1 is 16, so the semaphore will never be reduced to zero - enqueue will not block if the ring buffer is full - so overflow is possible.

```
void init() {
   p_m_init(&lock, NULL);
   sem_init(&s1, 0, 16);
   sem_init(&s2, 0, 0);
}
void *dequeue(){
   p_m_lock(&lock)
   sem_wait(&s2)
   void *result = b[(out++) & (N-1) ]
   sem_post(&s2)
   p_m_unlock(&lock)
   return result
}
```

- The initial value of s2 is zero, so calls to dequeue will always block and never return!
- The order of mutex lock and sem_wait will need to be swapped; however, this example is so broken that this bug has no effect!

```
void init() {
   sem_init(&s1,0,16);  sem_init(&s2,0,0);
}
enqueue(void *value){
   sem_wait(&s2);
   p_m_lock(&lock);
   b[ (in++) & (N-1) ] = value;
   p_m_unlock(&lock);
   sem_post(&s1);
}
void *dequeue(){
   sem_wait(&s1);
   p_m_lock(&lock)
   void *result = b[(out++) & (N-1)];
   p_m_unlock(&lock);
   sem_post(&s2);
   return result;
}
```

Another **incorrect** attempt

- The initial value of s2 is 0. **Enqueue will block on the first call** to sem_wait even though the buffer is empty!
- The initial value of s1 is 16. Dequeue **will not block on the first call** to sem_wait even though the buffer is empty - Underflow! The dequeue method will return invalid data.
- The code does not satisfy Mutual Exclusion. Two threads can modify in or out at the same time! The code appears to use mutex lock. Unfortunately, the lock was never initialized with pthread_mutex_init() or PTHREAD_MUTEX_INITIALIZER - so the lock may not work (pthread_mutex_lock may simply do nothing)

Correct implementation: enqueue

```
#include <pthread.h>
#include <semaphore.h>
#define N (16) // N must be 2^i
void *b[N]
int in = 0, out = 0
p_m_t lock = PTHREAD_MUTEX_INITIALIZER;
sem_t countsem, spacesem;
void init() {
   sem_init(&countsem, 0, 0);
   sem_init(&spacesem, 0, 16);
}
```

```
enqueue(void *value){
   // wait if there is no space left:
   sem_wait( &spacesem )

   p_m_lock(&lock)
   b[ (in++) & (N-1) ] = value
   p_m_unlock(&lock)

   // increment the count of the number of items
   sem_post(&countsem)
}
```

Correct implementation: dequeue

```c
#include <pthread.h>
#include <semaphore.h>
#define N (16) // N must be 2^i
void *b[N]
int in = 0, out = 0
p_m_t lock = PTHREAD_MUTEX_INITIALIZER;
sem_t countsem, spacesem;
void init() {
   sem_init(&countsem, 0, 0);
   sem_init(&spacesem, 0, 16);
}
```

```c
void *dequeue(){
   // Wait if there are no items in the buffer
   sem_wait(&countsem);

   p_m_lock(&lock);
   void *result = b[(out++) & (N-1)];
   p_m_unlock(&lock);

   // Increment the count of the number of spaces
   sem_post(&spacesem);
   return result
}
```

Exercises

- What would happen if the order of **sem_wait** and **pthread_mutex_lock** calls were swapped?

- What would happen if the order of **pthread_mutex_unlock** and **sem_post** calls were swapped?

```
void *dequeue(){
    // Wait if there are no items in the buffer
    sem_wait(&countsem);
    p_m_lock(&lock);
    void *result = b[(out++) & (N-1)];
    p_m_unlock(&lock);
    // Increment the count of the number of spaces
    sem_post(&spacesem);
    return result
}
```

Ring buffer as a consumer-producer with cvs

```
typedef struct {
   char buf[BSIZE];
   int occupied;
   int in;              /*index of in*/
   int out;             /*index of out*/
   pthread_mutex_t mutex;
   pthread_cond_t more; /*there is item*/
   pthread_cond_t less;    /*there is room*/
} buffer_t;
```

```
void enqueue(buffer_t *b, char item){
   pthread_mutex_lock(&b->mutex);
   while (b->occupied >= BSIZE)
       pthread_cond_wait(&b->less, &b->mutex);
   assert(b->occupied < BSIZE);
   b->buf[b->in++] = item;
   b->in &= BSIZE-1;
   b->occupied++;
   pthread_cond_signal(&b->more);
   pthread_mutex_unlock(&b->mutex);
}
```

https://docs.oracle.com/cd/E19455-01/806-5257/6je9h032r/index.html#sync-ex-33

```c
char dequeue(buffer_t *b){
   char item;
   pthread_mutex_lock(&b->mutex);
   while(b->occupied <= 0){
       pthread_cond_wait(&b->more, &b->mutex);
   }
   item = b->buf[b->out++];
   b->out &= BSIZE-1;
   b->occupied--;
   pthread_cond_signal(&b->less);
   pthread_mutex_unlock(&b->mutex);
   return(item);
}
```

Dequeue is a consumer

Enqueue is a producer

https://docs.oracle.com/cd/E19455-01/806-5257/6je9h032r/index.html#sync-ex-33

Deadlock

Deadlock describes a situation where two or more threads are blocked forever, waiting for each other.

Example

a. Thread 1 holds lock A and requests lock B
b. Thread 2 holds lock B and requests lock A

Deadlock occurs when

- Threads that are already holding locks request new locks
- The requests for new locks are made concurrently
- Two or more threads form a circular chain in which each thread waits for a lock which is held by the next thread in the chain

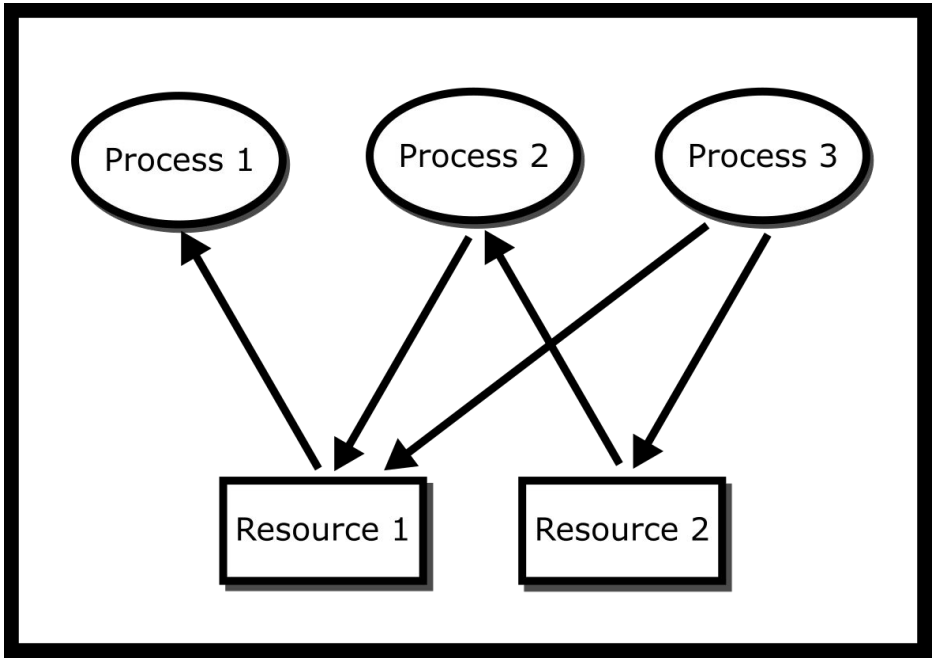
Livelock

A thread often acts in response to the action of another thread. If the other thread's action is also a response to the action of another thread, then *livelock* may result.

Starvation

Starvation describes a situation where a thread is unable to gain regular access to shared resources and is unable to make progress.

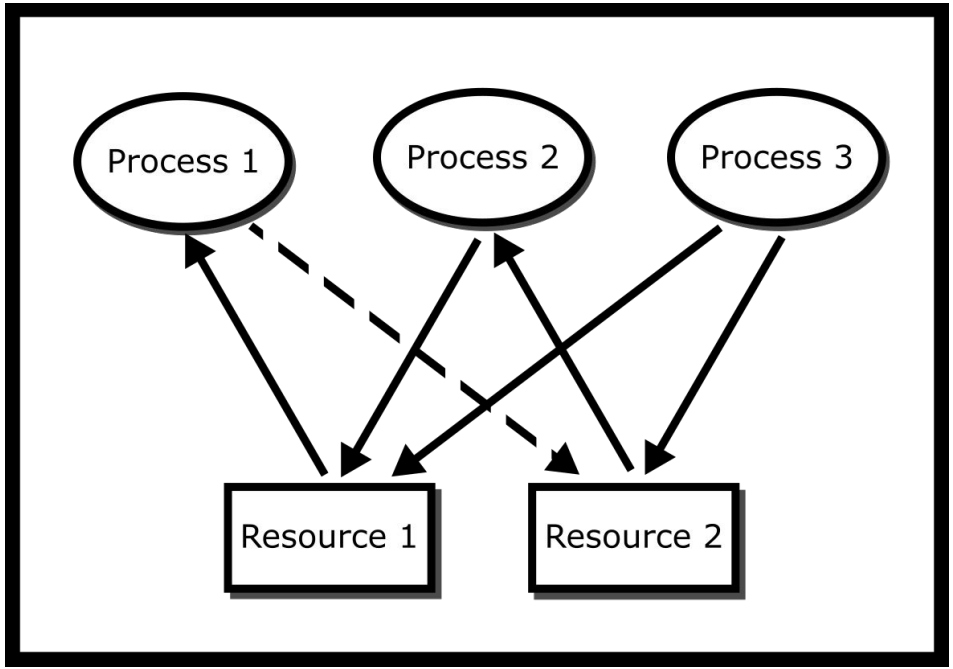
Deadlock: Resource allocation graphs



If there is a cycle in the Resource Allocation Graph and each resource in the cycle provides only one instance, then the processes will deadlock.

We can detect a deadlock by traversing the graph and searching for a cycle using a graph traversal algorithm, such as the Depth First Search (DFS).

Deadlock: Resource allocation graphs



If there is a cycle in the Resource Allocation Graph and each resource in the cycle provides only one instance, then the processes will deadlock.

We can detect a deadlock by traversing the graph and searching for a cycle using a graph traversal algorithm, such as the Depth First Search (DFS).

Coffman conditions

Deadlock can happen if and only if the four Coffman conditions are satisfied.

1. **Mutual Exclusion:** No two processes can obtain a resource at the same time.
2. **Circular Wait:** There exists a cycle in the Resource Allocation Graph:
   a. there exists a set of processes {P1,P2,. . . } such that P1 is waiting for resources held by P2, which is waiting for P3,. . . , which is waiting for P1.
3. **Hold and Wait:** Once a resource is obtained, a process keeps the resource locked.
4. **No Pre-emption:** Nothing can force the process to give up a resource.

Proof in the book

See also simulations in https://en.wikipedia.org/wiki/Deadlock#Individually_necessary_and_jointly_sufficient_conditions_for_deadlock

Approaches to solve deadlock

Ostrich algorithm:

ignore the deadlock, prevent processes from context-switching

"to stick one's head in the sand and pretend there is no problem".

Used in most operating systems

Banker's algorithm

https://en.wikipedia.org/wiki/Banker%27s_algorithm

- How much of each resource each process could possibly request ("MAX")
- How much of each resource each process is currently holding ("ALLOCATED")
- How much of each resource the system currently has available ("AVAILABLE")

Dining philosopher problem

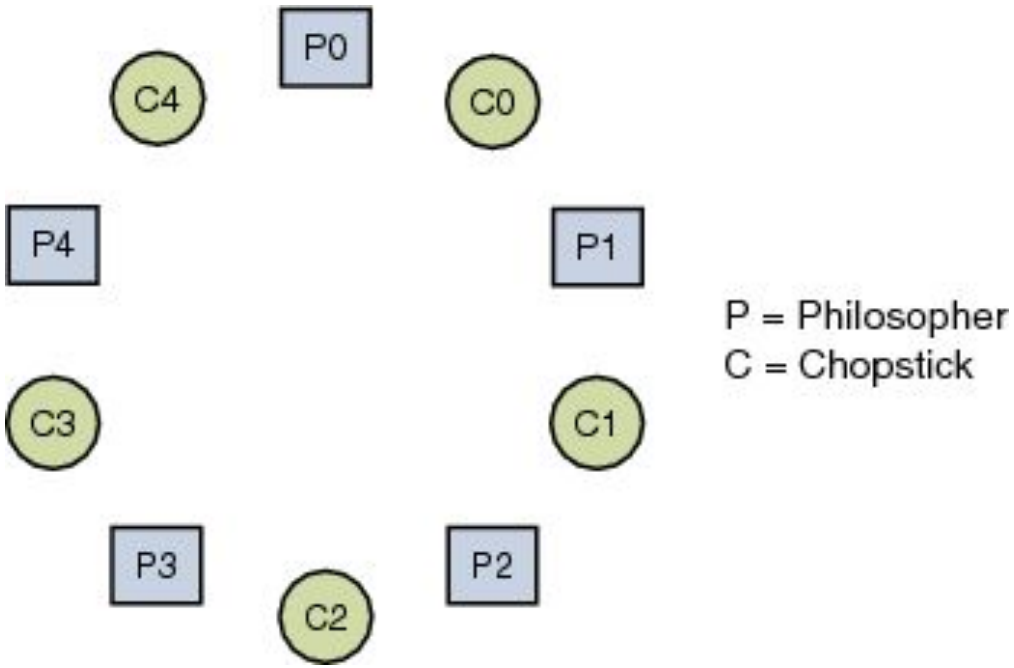
Five philosophers, numbered zero to four, are sitting at a round table, thinking.

As time passes, different individuals become hungry and decide to eat.

There is a platter of noodles on the table but each philosopher only has one chopstick to use.

In order to eat, they must share chopsticks.

The chopstick to the left of each philosopher (as they sit facing the table) has the same number as that philosopher.

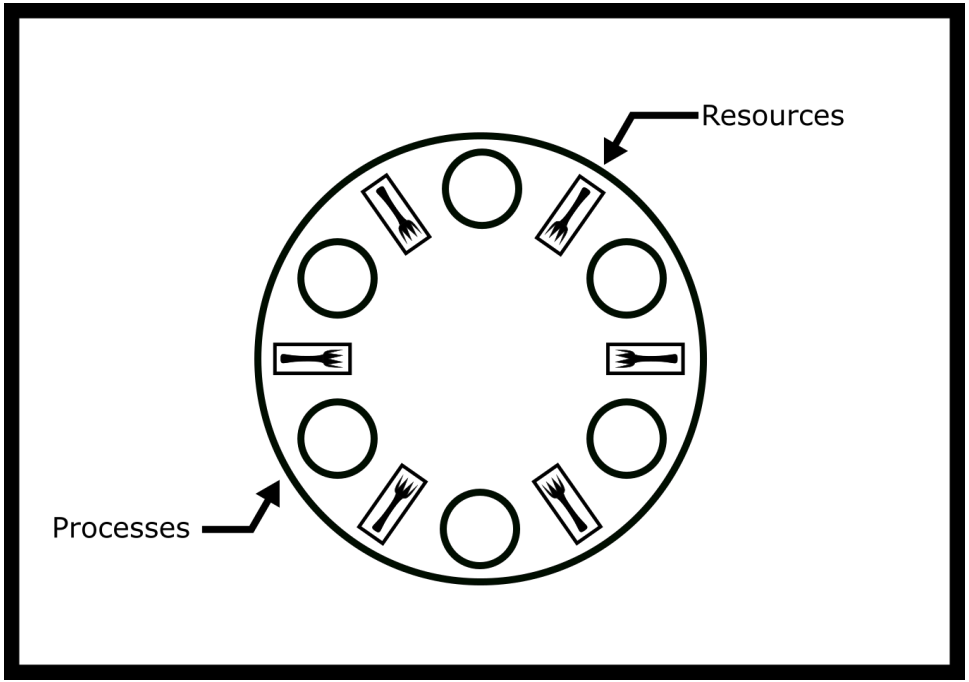


https://docs.oracle.com/cd/E19205-01/820-0619/geosb/index.html

- think until the left fork is available; when it is, pick it up
- think until the right fork is available; when it is, pick it up
- eat for a fixed amount of time
- put the right fork down
- put the left fork down
- think for a fixed amount of time
- repeat

Dining philosopher problem

```
philosopher(){
    while(true){
        think()
        pick(left_chopstick)
        pick(right_chopstick)
        eat()
        putdown_chopsticks()
    }
}
```

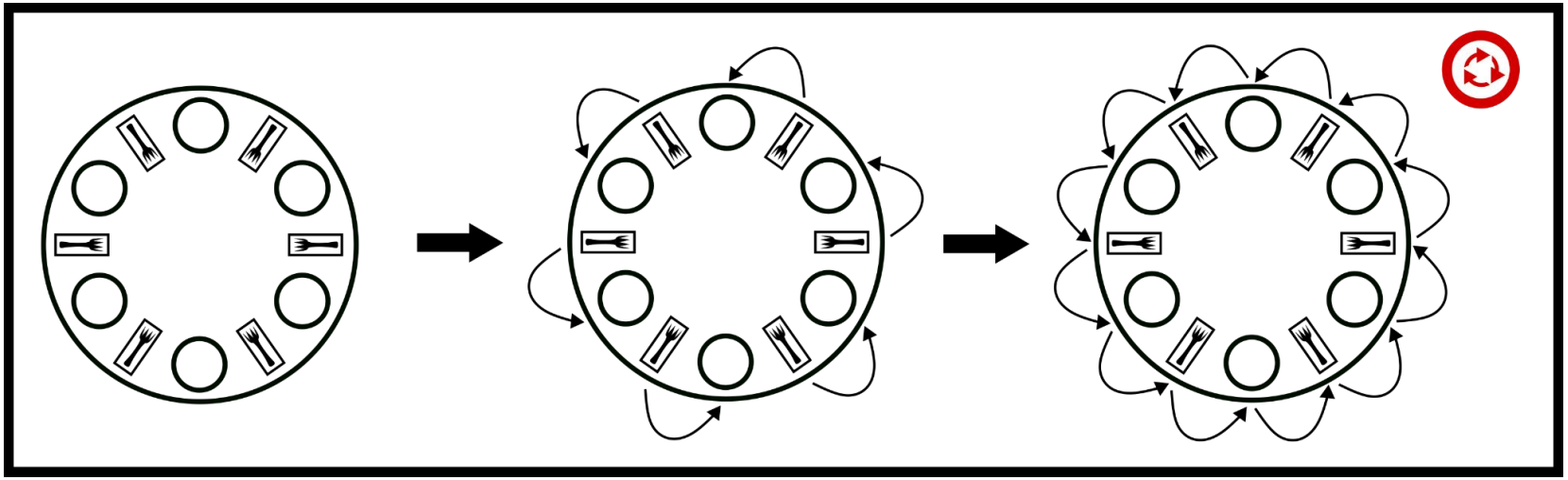


Dining philosopher problem

```
void *philosopher1(void *arg) {
 long i = (long)arg;
 pthread_mutex_t *left = &chopsticks[i];
 pthread_mutex_t *right = &chopsticks[(i + 1) % 5];
 while (1) {
   printf("id %ld: state: %.10s \n", i, "thinking!");
   printf("id %ld: state: %.10s \n", i, "hungry!");
   pthread_mutex_lock(left);
   pthread_mutex_lock(right);
   /*eat(left, right);*/
   printf("id %ld: state: %.10s \n", i, "eating");
   pthread_mutex_unlock(left);
   pthread_mutex_unlock(right);
   printf("id %ld: state: %.10s \n", i, "done!");
 }
}
```

How deadlock occurs?

What if everyone picks up their left chopstick and is waiting on their right chopstick?



resource allocation graph that shows how the system could be deadlocked

```c
void *philosopher2(void *arg) {
 long i = (long)arg;
 pthread_mutex_t *left = &chopsticks[i];
 pthread_mutex_t *right = &chopsticks[(i + 1) % 5];
 while (1) {
   /*think()*/
   int l = pthread_mutex_trylock(left);
   if (!l) { sleep(1);  continue;}
   int r = pthread_mutex_trylock(right);
   if (!r) {
     pthread_mutex_unlock(left);
     sleep(1);   continue;
   }
   // sleep(1); /*eat(left, right);*/
   printf("id %ld: state: %.10s \n", i, "eating");
   pthread_mutex_unlock(left);
   pthread_mutex_unlock(right);
   printf("id %ld: state: %.10s \n", i, "done!");
 }
```

Break hold and wait

scenario-1: If it's not available, they put the fork down and try again.

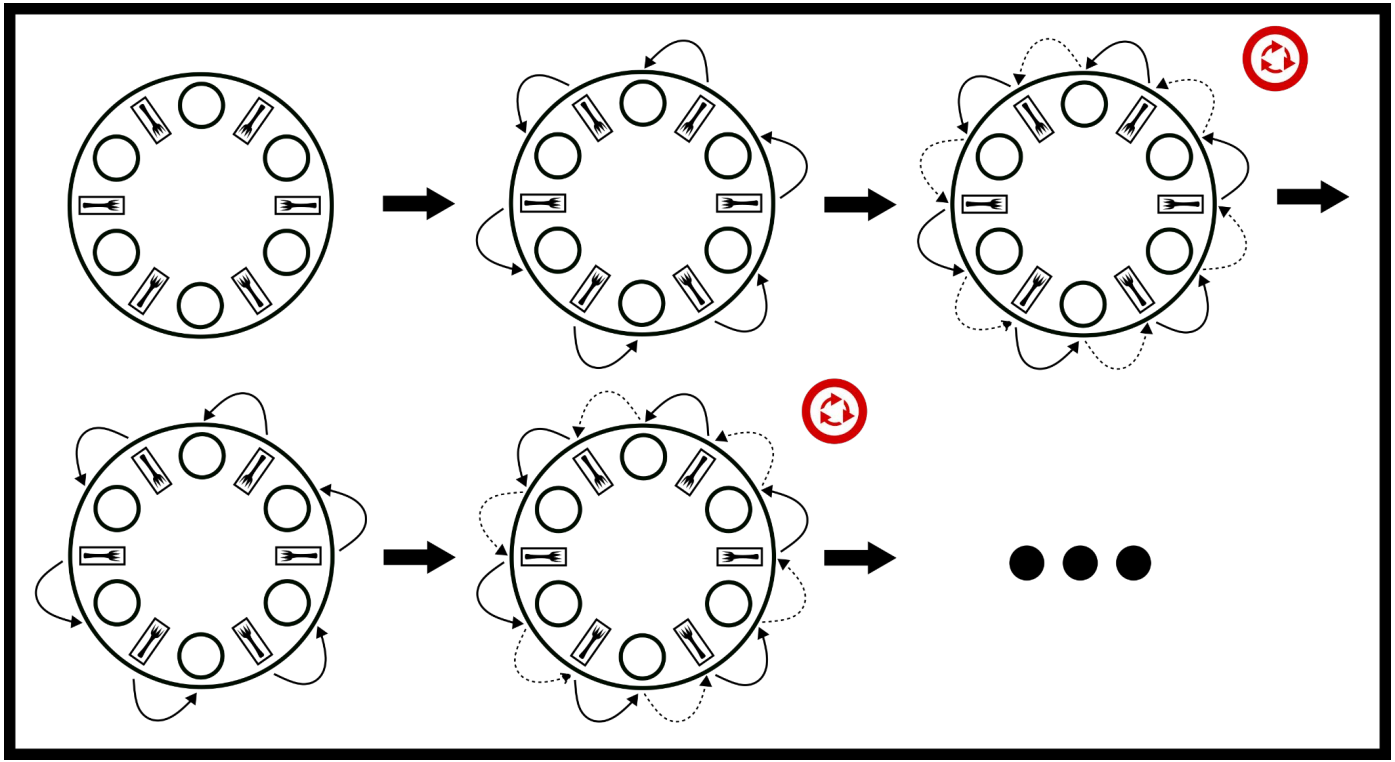
- **No deadlock in this scenario!**

scenario-2: all the philosophers pick up their left at the same time

- try to grab their right,
- put their left down,
- pick up their left, try to grab their right and so on.

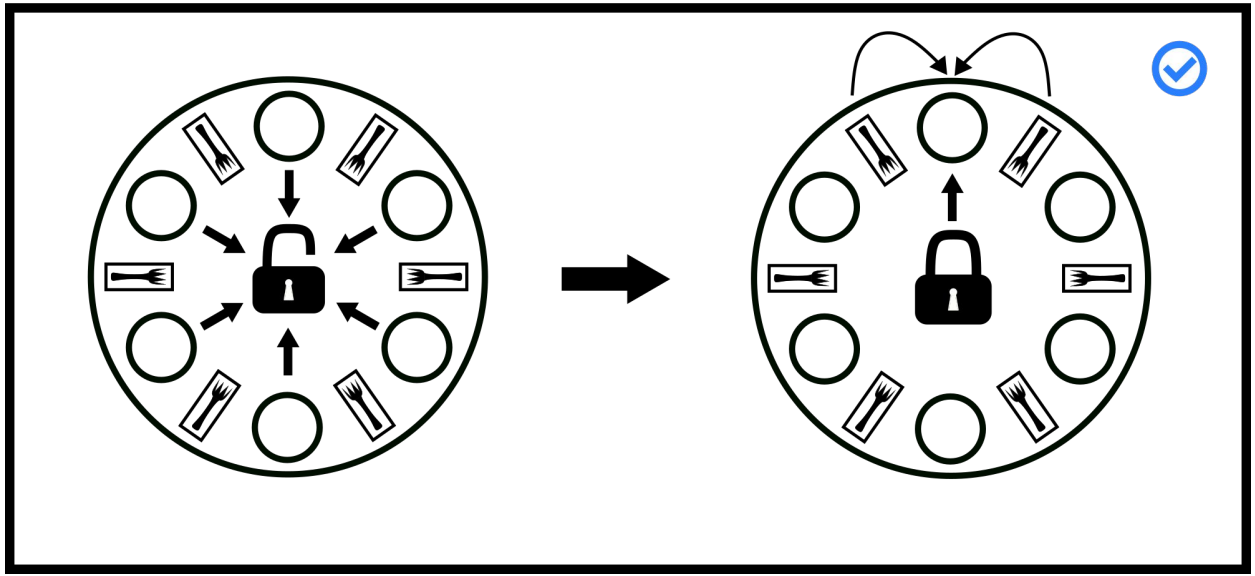
➔ **LIVELOCK**

LIVELOCK: a time evolution of the break hold and wait would look like.



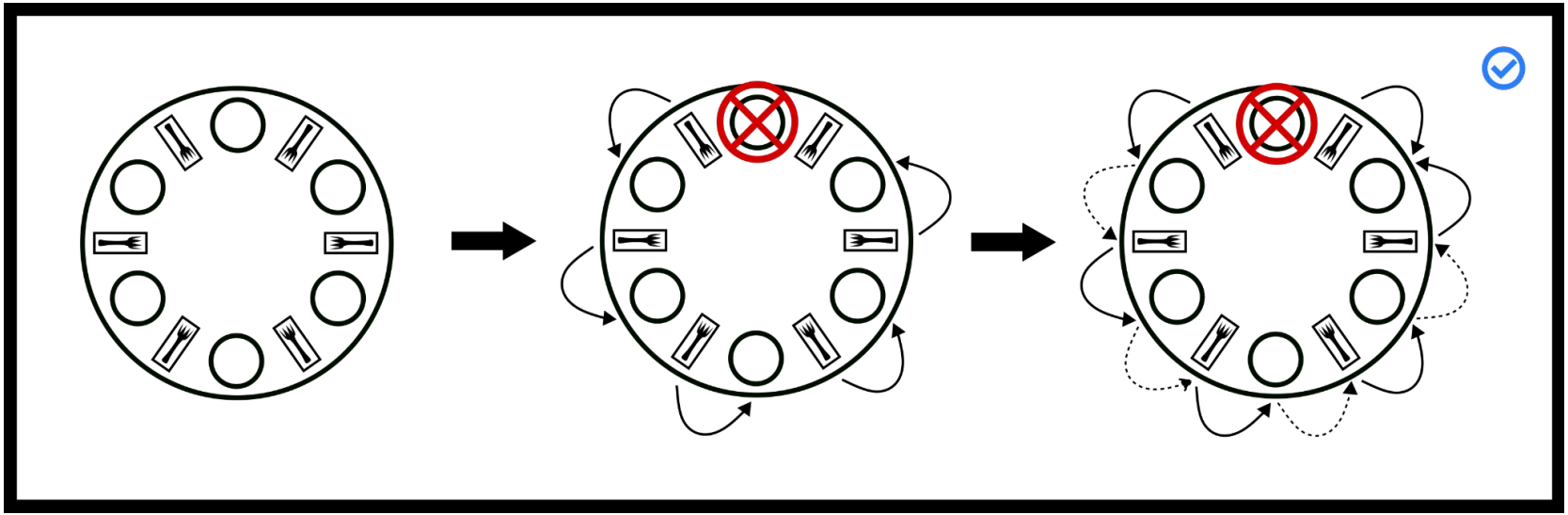
Arbitrator solution

- A philosopher can only pick up both or none by introducing an arbitrator, e.g., a waiter
  - Waiter can be implemented as a mutex
- Works but parallelism is reduced
  - All philosophers wait for waiter



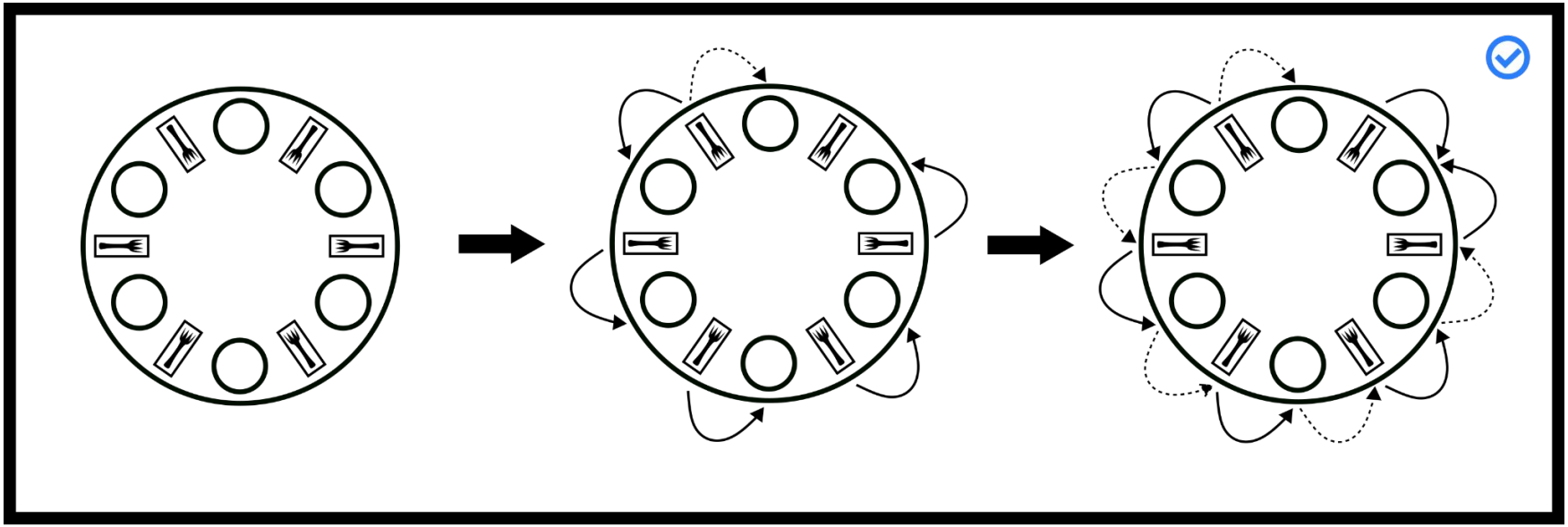
Stalling solution (leaving table)

Remove philosophers until deadlock is not possible. No circular wait means deadlock is not possible.



Partial (hierarchical) Ordering (Dijkstra's solution)

- Assign a number to each philosopher and chopstick
- A philosopher can choose two chopsticks
  - i,
  - i+1 %5
- Each philosopher takes **the lower numbered chopstick first**
  - P0-C0,
  - P2-C2,
  - P3-C3,
  - **P4-cannot take C0**
    - The highest numbered philosopher does not have any chopy stick.
- Then each gets the higher numbered chopstick
  - **P4 is able to get C5 and eat**

Partial (hierarchical) ordering (Dijkstra's solution)

Each process picks only lower numbered chopstick first, then the higher numbered chopstick.